

Multipattern Integrated Networks With Contrastive Pretraining for Graph Anomaly Detection

Manzhi Yang¹, Jian Zhang², Liyuan Lin², Jinpeng Han², *Student Member, IEEE*, Xiaoguang Chen¹, Zhen Wang², and Fei-Yue Wang², *Fellow, IEEE*

Abstract—As a challenge of practical significance, fraud detection has great potential for telecom fraud prevention, economic crime prevention, and personal property preservation. Fraudulent activities are always buried in massive regular transactions, making it hard to find them. Traditional rule-based approaches need multiple domain-specific rules and multistep verification, which limits their transferability and efficiency. Machine learning-based methods might ignore the intricate interactions or the temporal relations among accounts. Meanwhile, the lack of sufficient manual labels restricts their performance. To overcome the above limitations, we present a multipattern integrated network (MPIN) in this article to identify fraudulent accounts in transaction networks. Specifically, MPIN considers the interactions among nodes from three perspectives: inflows, outflows, and their mutual influences. To learn the behavior pattern of each node, MPIN first applies an attention mechanism to integrate the short-term information and then learns the long-term patterns by aggregating multiple short-term patterns. Behavior patterns from different perspectives together with long short-term modeling enable the model to precisely distinguish fraudulent accounts from the normal ones. Moreover, contrastive pretraining with temporal consistency and local tightness guarantee is adopted to alleviate the label sparsity issue and provide the model with low-variance performance. We conducted experiments on two real-world transaction networks, and the results showed the effectiveness of MPIN compared with five state-of-the-art baselines.

Index Terms—Anomaly detection, contrastive learning, graph neural network, network representation learning.

I. INTRODUCTION

NOWADAYS, services such as mobile communication, online payment, and e-commerce provide tremendous convenience in our daily lives. Such convenient services, however, also cause security issues in the meanwhile [1], [5], [7]. Not only normal users but also service providers themselves suffer from the losses and damages caused by deceptions [6]. Fraudulent activities pose a multifaceted threat, socially eroding public trust in institutions and economically inflicting billions in losses and increased operational costs while also compromising security by exploiting vulnerabilities in data [3] and financial systems. From communications both over regular terminals and Internet telephones to cryptocurrency exchanges in blockchain-based platforms, even the real money transactions in online payment services, frauds breed where profits exist. The critical importance of fraud detection lies in its ability to mitigate these social, economic, and security risks. Effective fraud detection strategies are essential to safeguard assets, maintain public confidence, and ensure the smooth functioning of financial systems, making it a key area of focus in both the private and public sectors. Hence, it is of practical significance to find out those who are involved in fraudulent cases to eliminate financial losses and even prevent fraud from happening in advance.

In the early exploration of fraud detection, rule-based detection systems [8] are widely deployed due to its efficiency and effectiveness. Practitioners summarize fraudulent patterns and then apply heuristic rules to filter out fraudulent activities as well as related fraudsters. Thus, the decisions [4] provided by rule-based detection systems are explainable and can greatly save the cost of manual reviews. Benefiting from their effectiveness and simplicity, rule-based approaches still play important roles in industrial scenarios. For instance, in banking transactions, transactions that involve abnormal amounts of currency or relate to risky accounts would be intercepted by the monitoring systems. Another typical example is telecommunication fraud, shown in Fig. 1. The victim may be involved in a series of phone frauds that are hidden in numerous normal communication records. It is hard for a bunch of handcraft-designed rules to find the fraud. Despite the above

Manuscript received 2 September 2023; revised 18 January 2024; accepted 31 January 2024. Date of publication 18 July 2024; date of current version 2 October 2024. This work was supported by the Science and Technology Development Fund, Macao SAR under Grant 0093/2023/RIA2 and the National Natural Science Foundation of China under Grant 62176080. (Manzhi Yang and Jian Zhang contributed equally to this work.) (Corresponding author: Fei-Yue Wang.)

Manzhi Yang and Xiaoguang Chen are with Macau Institute of Systems Engineering, Macau University of Science and Technology, Macao 999078, China (e-mail: yangmanzhi@eversec.cn; chenxiaoguang@eversec.cn).

Jian Zhang and Liyuan Lin are with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310000, China (e-mail: zhangjian-hdu@hdu.edu.cn; linliyuan@hdu.edu.cn).

Jinpeng Han is with the School of Software Engineering, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: jinpeng.han@stu.xjtu.edu.cn).

Zhen Wang is with the School of Cyberspace and Experimental Center of Data Science and Intelligent Decision-Making, Hangzhou Dianzi University, Hangzhou 310000, China (e-mail: wangzhen@hdu.edu.cn).

Fei-Yue Wang is with the State Key Laboratory for Management and Control of Complex Systems and Beijing Engineering Research Center of Intelligent Systems and Technology, Chinese Academy of Sciences, Beijing 100190, China, also with Macau Institute of Systems Engineering, Macau University of Science and Technology, Macao 999078, China, and also with Beijing Huairou Academy of Parallel Sensing, Beijing 101407, China (e-mail: feiyue.wang@ia.ac.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TCSS.2024.3362393>, provided by the authors.

Digital Object Identifier 10.1109/TCSS.2024.3362393

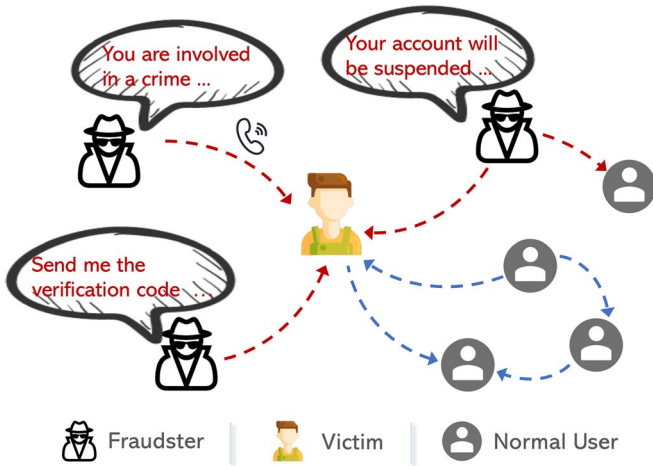


Fig. 1. Example of telecom fraud.

advantages, rule-based detection systems are usually a summary of expert experiences provided by game operators, making it difficult to deal with unknown and frequently evolving fraud patterns.

After collecting a large amount of data from online services, such as transaction information and fraud labels, fraud identification could also be addressed by machine learning-based models. Considering the behaviors of an account as a sequence of events, time-series analysis could be applied to find the outliers that could be regarded as potential frauds. However, such methods only focus on the individual patterns of fraud and ignore the fraudulent collaboration between group members. Recently, some researchers proposed to model the transactions as networks and argued that exploiting the network structure could help with fraud detection [9]. For example, a bunch of graph-based algorithms is proposed to detect credit card frauds either in real-world scenarios [10] or in cryptocurrency transactions [11].

Although great efforts have been devoted to fraud detection, it is still challenging for real-world applications due to the following reasons.

- 1) *Label scarcity and class imbalance*: Machine learning-based models often require sufficient fraud labels to ensure performance, while fraud labels are always scarce in real-world scenarios. In telecom fraud detection, most fraudulent phone numbers are recognized based on reports from victims, and so are the fraudsters in other scenarios. Such manual annotation is costly and labor-consuming in real-world applications. At the same time, fraud labels are extremely unbalanced since fraudsters only account for a small percentage of regular accounts. As a result, the lack of sufficient and balanced labels makes it difficult to train classification models in a supervised manner, which may lead to problems such as over-fitting and unstable performances [1].
- 2) *Group collaboration and temporal correlation*: Fraudsters in practical scenarios do not commit fraud separately. They often work as a team, which makes the fraudulent patterns intricate and implicit. In telecom

frauds, several phone numbers call one victim to make up a specific scenario for the victim to commit fraud. Similarly, chainlike transaction in financial scenarios is also a pattern of group collaboration. Besides, such collaborations are not finished immediately but form temporal correlation, which could not be characterized by static networks.

To address the above challenges, we propose a multipattern integrated network (MPIN) to identify fraudulent accounts purely based on network structures. Modeling the interactions between entities as a temporal network, MPIN pays attention to the directions of entity interactions and considers three different views, i.e., *inflows*, *outflows*, and their *mutual influences*, to learn interaction patterns. Inflows of a node are the interactions initiated by other nodes, while outflows of a node represent the interactions started by the node on its initiative. Incoming calls and outgoing calls could be the representatives of inflows and outflows. Meanwhile, we assume that each in-edge of a node is associated with its out-edges and vice versa. An intuitive example is that the payment of an account is direct. Therefore, the mutual influence of inflows and outflows should be considered when modeling node interaction patterns. Furthermore, considering the dynamics of the network evolving, we divide the whole network into snapshots at a given timespan Δt and merge the short-term behavior patterns into long-term patterns. To compensate for the lack of sufficient supervised signals caused by label scarcity and class imbalance, contrastive pretraining with two pretext tasks is designed to enable the model to learn more informative representations by self-discrimination from massive unlabeled data. The pretext tasks are designed based on two assumptions: temporal consistency that the change of time window has little influence on one's behavior pattern; and local tightness that individuals with similar interaction patterns should be gathered. In summary, our main contributions are listed as follows.

- 1) We propose an end-to-end framework, namely MPIN, to identify the anomalous nodes in graphs. Jointly considering the directions of node interactions and temporal correlations between them, the framework learns the long short-term node behavior patterns for fraud detection.
- 2) Contrastive pretraining with two pretext tasks is implemented to initialize model parameters by self-discrimination. The pretraining process, which ensures temporal consistency and local tightness, effectively alleviates the label scarcity problem and paves the way for behavior pattern learning with better generalization.
- 3) We conduct experiments on two networks extracted from real-world scenarios, i.e., telecommunication and cryptocurrency platforms. The experimental results show the superior performance of the MPIN compared with the several state-of-the-art baselines. Then, the ablation studies verify the effectiveness of each model component. Finally, elaborate analysis and visualization shed light on the effectiveness of the MPIN.

The rest of the article is organized as follows. First, we briefly review the literature about fraud detection, graph representation learning, and contrastive learning in Section II. Then,

Section III gives a detailed illustration of the proposed framework, and Section IV presents comprehensive experiments together with the discussion of the results. Finally, we conclude the article in Section V.

II. RELATED WORK

This section briefly reviews the research on fraud detection, graph representation learning, and contrastive learning.

A. Fraud Detection

Frauds could be considered as a kind of anomaly in real-world scenarios such as credit card transactions [12], telecommunication, and news spreading [13]. Machine learning-based methods focusing on fraud detection could roughly be divided into feature classification [14], sequence analysis [15], and graph-based models [16]. In credit card fraud detection, features such as transaction ID and transaction amount should be extracted before applying machine learning methods [14], [17]. Fraud detection is a typical imbalanced classification problem. Han et al. [18], [19] applied multimodal multiobjective optimization to detect credit card fraud. Noisy-sample-removed under-sampling scheme (NUS) [20] to improve classification performance in imbalanced datasets by removing noisy samples and using logistics regression, decision tree, and random forest as base classifiers. Weighted extreme learning machine [21] could also be used to solve the problem. Frauds could also be detected based on sequence feature learning. DIFM [22] captures information on field value variations and field interactions of user activities on e-commerce platforms with a factorization machine. The importance-aware module makes the model an explainable one for fraud detection. Besides, there are also many other researches [23], [24] attempt to identify frauds based on time-series features. Graph-based fraud detection methods model the interaction between entities as graphs and then find graph anomalies. For example, GEM [25] constructs heterogeneous graphs of devices and activities and applies an attention-based graph neural network to detect malicious accounts. HGAT [26] leverages high-order graph attention representations for loan risk assessment. Focusing on loan frauds, GRC [27] integrates a role-constrained conditional random field into a graph neural network to make use of role information and aims to detect collusive fraud in consumer loans.

B. Graph Representation Learning

Graph representation learning tries to map nodes in networks from sparse high-dimensional space to dense lower dimensional representations, making it possible to apply machine learning models in subsequent tasks. Starting from random walk-based methods such as DeepWalk [28] and node2vec [29] to graph neural networks with the representative of GCN [30], GAT [31], and GraphSage [32], a vast of research has been devoted to learning expressive node representations on static networks. A bunch of models developed upon GCN such as cluter-GCN [33] and deepGCN [34] were explored to achieve fast network

structure learning or deepen the model for more expressive representations.

To date, dynamic networks also have gained much more research interests recently [35], [36]. Temporal walk [37], [38] extends random walk to dynamic networks to learn both structural and temporal information from node sequences in chronological order. Considering the dynamics of the network evolving, DyRep [39] models two kinds of events as two interleaved Hawkes processes and learns node embedding with an attention mechanism. Other methods [40], [41] that focus on discrete dynamic networks are also developed to meet different practical needs. Though the above algorithms are effective in their fields, the latent anomalous behavior patterns are still hard to capture. Moreover, the lack of enough labeled data further makes it harder for them to make precise judgments.

C. Contrastive Learning

As a kind of self-supervised learning, contrastive learning captures internal data features by maximizing the agreement of similar samples without labels. With the success in computer vision [42], [43] and natural language processing [44], contrastive learning is also introduced into graph representation learning. Following the idea of Deep InfoMax [45], DGI [46] maximizes the mutual information between patch representations and corresponding high-level summaries of graphs to learn graph representations. Improvement on heterogeneous networks [47] and temporal networks [48] based on DGI are proposed for specific scenarios. The main challenge for contrastive learning is to construct proper positive samples. GraphCL [49] summarizes four types of graph data augmentation, which shows their effectiveness in GNN pretraining. CoLA [50] adopts subgraph embeddings as the positive sample for target node to learn node representations. Graph representation in another latent space could also serve as positive samples [51]. CG³ [52] utilizes both localized and hierarchical graph convolutions to generate multiview graph representations and then implements contrastive learning to train the model. Many other researchers [46], [53] also employ contrastive learning on graphs to avoid over-fitting and obtain more robust representations.

III. METHOD

A. Problem Formulation

Suppose we have a network $\mathcal{G} = (\mathcal{V}, \mathcal{E}_T, X)$ where $\mathcal{V}$ denotes the node set and $\mathcal{E}_T$ signifies the temporal edges with timestamps. Each edge $e = (u, v, t) \in \mathcal{E}_T$ is associated with a source node $u \in \mathcal{V}$, a target node $v \in \mathcal{V}$, and a timestamp $t \in \mathbb{R}^+$ representing the interaction time. The node feature is denoted by $\mathcal{X} \in \mathbb{R}^{|\mathcal{V}| \times D}$ where D represents the feature dimension. It is worth noticing that there could be multiple interactions between two nodes at different times, which makes $\mathcal{G}$ a multidigraph. In the multidigraph, each node is annotated with a label y to show whether it is a fraudulent node or not. In this article, we aim to identify whether a given node u is anomalous or not, which could be expressed $p(y_u = 1|u, \mathcal{G})$. Specifically, we innovatively divide the original dynamic graph with a given

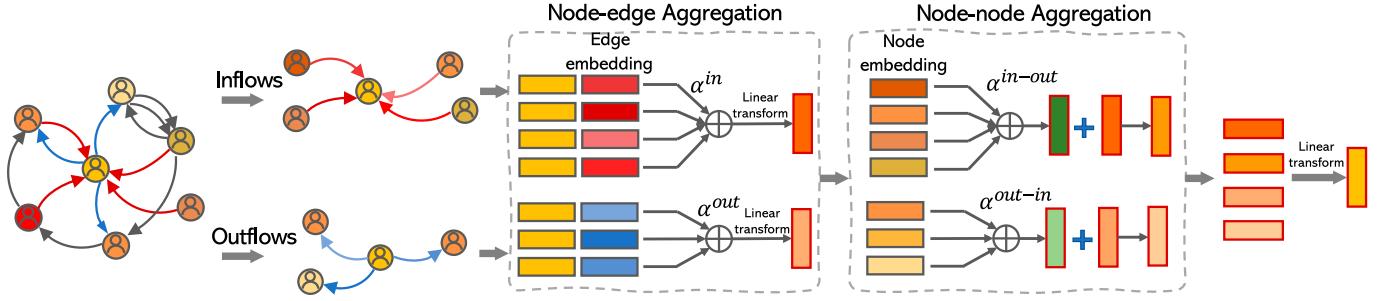


Fig. 2. Illustration of short-term behavior pattern modeling with multiview attention.

timespan Δt and then form a series of snapshots, denoted by $S_G = \{\mathcal{G}_1, \dots, \mathcal{G}_n\}$. Each snapshot $\mathcal{G}_k$ could also be considered as a dynamic graph to characterize the short-term behavior patterns. Finally, the goal of this article could be expressed as $p(y_u = 1|u, S_G)$.

B. Behavior Pattern Modeling

The behaviors act as *fingerprints* of entities, informing us which kind of entity it might belong to. In online transactions, active accounts continuously trade with others, while fallow accounts might have transaction records only within a short space of time. In telecommunications, fraudsters are intended to call a large proportion of unacquainted phone numbers, while the contacts of normal users are usually within a small amount of familiar phone numbers. However, the differences between fraudulent nodes and normal ones are nontrivial. Fraudulent behaviors are always buried in numerous normal records, making them hard to identify. In this article, we propose to learn both short-term and long-term behavior patterns for each account to distinguish fraudulent nodes from normal ones.

1) *Short-Term Behavior Pattern*: It characterizes one's behavior pattern in a short observation time Δt , which could reflect its intentions in the short run. For instance, a beginner in online games would like to frequently buy more in-game equipment to upgrade or gain special powers. While such intensive transactions might decrease as the player gets stronger. Hence, proper modeling of the short-term behavior pattern could benefit the identification of the roles of different accounts. Given a snapshot dynamic graph observed in Δt , as we have mentioned, for node i in the graph, its behavior could be depicted from three aspects: inflows, outflows, and their mutual influence. Mathematically, the inflow pattern could be represented as

$$\mathbf{h}_i^{\text{in}} = \sum_{(j,i,t) \in \mathcal{E}_T} \alpha_{j,i,t}^{\text{in}} (\mathbf{h}_i || \mathbf{h}_{j,i,t})$$

$$\alpha_{j,i,t}^{\text{in}} = \frac{\exp(\mathbf{W}_{\text{trans}}^{\text{in}}(\mathbf{h}_i || \mathbf{h}_{j,i,t}))}{\sum_{(j,i,t) \in \mathcal{E}_T} \exp(\mathbf{W}_{\text{trans}}^{\text{in}}(\mathbf{h}_i || \mathbf{h}_{j,i,t}))} \quad (1)$$

where $\mathbf{W}_{\text{trans}}^{\text{in}} \in \mathbb{R}^{D \times D'}$ is the trainable weight for attention between node i and its in-edges and D' represents the output dimension of the attention module. $\alpha_{j,i,t}^{\text{in}}$ is the obtained attention weight over node i and its temporal in-edge (j, i, t) . In the

same way, we could also obtain the representation of outflow pattern $\mathbf{h}_u^{\text{out}}$

$$\mathbf{h}_i^{\text{out}} = \sum_{(i,j,t) \in \mathcal{E}_T} \alpha_{i,j,t}^{\text{out}} (\mathbf{h}_i || \mathbf{h}_{i,j,t})$$

$$\alpha_{i,j,t}^{\text{out}} = \frac{\exp(\mathbf{W}_{\text{trans}}^{\text{out}}(\mathbf{h}_i || \mathbf{h}_{i,j,t}))}{\sum_{(i,v,t) \in \mathcal{E}_T} \exp(\mathbf{W}_{\text{trans}}^{\text{out}}(\mathbf{h}_i || \mathbf{h}_{i,v,t}))} \quad (2)$$

where $\mathbf{W}_{\text{trans}}^{\text{out}} \in \mathbb{R}^{D \times D'}$ is trainable weight for attention between node i and its out-edges and $\alpha_{j,i,t}^{\text{out}}$ is the obtained attention weight over node i and its temporal out-edge (i, j, t) .

Considering inflows and outflows of an account separately could provide insights of the scale and volatility of one's fortune. Meanwhile, the relation between each inflow and outflow also matters. Despite the fortune obtained by topping up with real money, the profits earned through trading are another major source of assets. It means that the money one spends has latent relations with the money he/she earns. Hence, we further model such relationships as follows:

$$\mathbf{h}_u^{\text{in-out}} = \sum_{i \in \mathcal{N}_u^{\text{out}}} \alpha_{ui}^{\text{in-out}} \mathbf{h}_i^{\text{in}}[n]$$

$$\alpha_{ui}^{\text{in-out}} = \frac{\exp(\mathbf{h}_u^{\text{in}} \mathbf{h}_i^{\text{in}})}{\sum_{j \in \mathcal{N}_u^{\text{out}}} \exp(\mathbf{h}_u^{\text{in}} \mathbf{h}_j^{\text{in}})} \quad (3)$$

where $\mathbf{h}_u^{\text{in-out}}$ represents the influence that the inflows of u have on the outflows. Similarly, $\mathbf{h}_u^{\text{out-in}}$ denotes the influence that the outflows of u have on the inflows. The short-term behavior pattern extraction scheme is presented in Fig. 2.

2) *Long-Term Behavior Pattern*: By now, we have obtained three types of short-term behavior patterns reflecting an individual's trading behaviors in a short period of time. Nevertheless, the transaction features in the short run are insufficient to distinguish one from the others. To characterize one's behavior pattern in the long run, we assemble all the short-term patterns. Take the learning of long-term inflow pattern, for instance, we first sort the short-term inflow patterns of u to obtain $\mathbf{h}_u^{\text{in}}$ in chronological order and apply GRU, a variation of RNN, to learn the transition patterns among different intervals

$$\mathbf{h}_u^{\text{in}} = \text{GRU}([\mathbf{h}_u^{\text{in}}[1], \dots, \mathbf{h}_u^{\text{in}}[n]]). \quad (4)$$

The long-term pattern of outflow and mutual influence, denoted by $\mathbf{h}_u^{\text{out}}$ and $\mathbf{h}_u^m$, respectively, could also be obtained in the same way. These three patterns describe an account from different

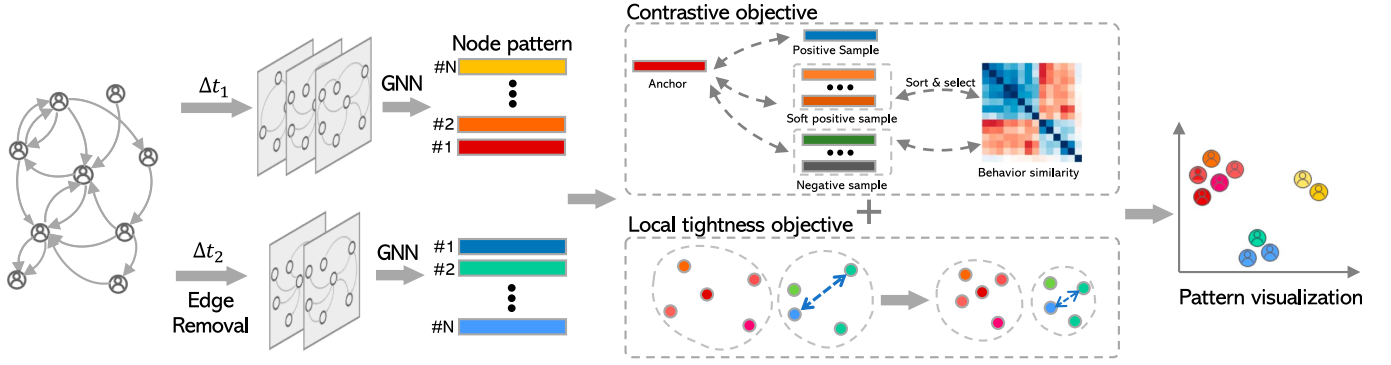


Fig. 3. Illustration of contrastive pretraining.

aspects, and we merge them by concatenation as the final representation $\mathbf{h}_u \in \mathbb{R}^{3D'}$

$$\mathbf{h}_u = \text{Merge}(\mathbf{h}_u^{\text{in}}, \mathbf{h}_u^{\text{out}}, \mathbf{h}_u^m). \quad (5)$$

C. Learning With Contrastive Pretraining

Traditional supervised learning requires large amounts of labeled data to ensure fair performance. However, fraudulent accounts are hard to annotate in real-world applications. Thus, we propose to adopt contrastive learning to pretrain the foregoing behavior pattern extraction module, and the procedure is presented in Fig. 3. The goal of contrastive pretraining is to make sure that each representation is distinguishable compared with the others, and thus the model could learn distinct behavior patterns. Following the general scheme of contrastive learning, we need to find a positive sample, soft positive samples, and negative samples for the target node at first.

1) *Positive Sample*: Under the assumption that the behavior patterns of one account observed under different Δt should be identical, we change Δt to $\Delta t'$ to obtain a new graph sequence $S_{G'}$. To increase the local structure diversity of accounts, we further randomly remove $\beta|\mathcal{E}_T|$ transactions from G' where $\beta \in (0, 1)$ represents the pruning rate. Based on $S_{G'}$, we could obtain $\mathbf{h}'$, which can be regarded as the positive sample of $\mathbf{h}$, for each account following the above behavior pattern extraction procedure.

2) *Soft Positive Sample*: In addition to constructing a positive sample for each node, we propose to use the nodes with similar behavior features compared with the target node, which are considered as soft positive samples, to further regularize the learning of node representations. Specifically, we first calculate the behavior similarity by $\mathcal{X}\mathcal{X}^T$ and then select k_{sp} nodes as the soft positive samples for each node after descending sort. The selected nodes are denoted by $\mathcal{V}^{\text{sp}}$.

3) *Negative Sample*: We then select the last k_{neg} nodes, denoted by $\mathcal{V}^{\text{neg}}$, that are different from the target node as the negative samples. Apparently, with u as the target node we need to draw the distance between h_u and h'_u close and widen the gap between $\mathbf{h}_u$ and $\mathbf{h}_j$, where $j \in \mathcal{V}_u^{\text{neg}}$.

Hence, the contrastive objective could be modeled in the form of InfoNCE [54], [55]

$$\mathcal{L}_c = -\frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \log \frac{\exp(\frac{1}{\tau} \mathbf{h}_i^T \mathbf{h}'_i) + \sum_{j \in \mathcal{V}_i^{\text{sp}}} \exp(\frac{1}{\tau} \mathbf{h}_i^T \mathbf{h}'_j)}{\exp(\frac{1}{\tau} \mathbf{h}_i^T \mathbf{h}'_i) + \sum_{j \in \mathcal{V}_i^{\text{sp}} \cup \mathcal{V}_i^{\text{neg}}} \exp(\frac{1}{\tau} \mathbf{h}_i^T \mathbf{h}'_j)} \quad (6)$$

where τ represents the temperature parameter.

Apart from the need for temporal consistency, we would like the accounts with similar behavior patterns to be clustered together to ensure local tightness. For account i , we first measure its behavior pattern $\mathbf{h}_i$ with other accounts with a score function $s(\mathbf{h}_i, \mathbf{h}_j) = \text{cosine}(\mathbf{h}_i, \mathbf{h}_j)$ and then filter out the top k_l accounts, denoted by $\mathcal{V}_i^{\text{top}}$, that share the most similar behavior patterns with i . To ensure the local tightness, we force h_i to be close to the local center $\bar{\mathbf{h}}_i$, and the optimization goal $\mathcal{L}_t$ is modeled as

$$\mathcal{L}_t = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \|\mathbf{h}_i - \bar{\mathbf{h}}_i\|_2$$

$$\bar{\mathbf{h}}_i = \frac{1}{|\mathcal{V}_i^{\text{top}}| - 1} \sum_{j \in \mathcal{V}_i^{\text{top}}} \mathbf{h}_j. \quad (7)$$

Finally, we obtain the loss function for pretraining

$$\mathcal{L}_p = (1 - \lambda)\mathcal{L}_c + \lambda\mathcal{L}_t \quad (8)$$

where $\lambda \in [0, 1]$ is a hyperparameter used for balancing $\mathcal{L}_c$ and $\mathcal{L}_t$. Finally, the overall training procedure of MPIN is shown in Algorithm 1.

D. Malicious Account Detection

After obtaining the behavior patterns for each account, we then apply graph convolution layers to make the final prediction. Denoting the graph convolution layer applied on a single node as f , the calculation could be described as

$$f(\mathbf{h}_u) = \text{ReLU} \left(\frac{1}{|\mathcal{N}_u| + 1} \sum_{i \in \mathcal{N}_u \cup \{u\}} \mathbf{W}_o^T \mathbf{h}_i + \mathbf{b} \right) \quad (9)$$

where $\mathbf{W}_o \in \mathbb{R}^{F \times 3D'}$ are trainable weights and $\mathbf{b} \in \mathbb{R}^F$ represents the bias. $\mathcal{N}_u$ represents both in-neighbors and out-neighbors of u . Multiple graph convolution layers are applied

Algorithm 1 Training Procedure of *MPIN*

Input: Transaction network $\mathcal{G}$, observation interval Δt and $\Delta t'$, the number of soft positive samples k_{sp} , the number of negative samples k_{neg} , model's parameter Θ

Output: Prediction results $\{\hat{y}_i\}$, model $\mathcal{M}$

Initialize an MPIN model $\mathcal{M}$ with Θ .
 Slice $\mathcal{G}$ at the interval of Δt to obtain $S_{\mathcal{G}}$;
 # Contrastive pre-training.
repeat
 Select k_{sp} nodes as soft positive samples and k_{neg} nodes as negative samples for each node ;
 Slice $\mathcal{G}$ at the interval of $\Delta t'$ to obtain $S_{\mathcal{G}'}$;
 Obtain $\mathbf{h}$ for each node in $\mathcal{G}$ based on Eq. (1)-(5);
 Obtain $\mathbf{h}'$ for each node in $\mathcal{G}'$ based on Eq. (1)-(5);
 Calculate pair-wise similarity among all nodes and obtain k_c nodes for each node as a cluster;
 Calculate $\mathcal{L}_p$ based on Eq. (6)-(8);
 Update $\mathcal{M}$'s parameter Θ .
until converge
 # Supervised learning.
repeat
 Obtain $\hat{y}$ for each node based on Eq. (9)-(10);
 Calculate $\mathcal{L}$ based on Eq. (11);
 Update $\mathcal{M}$'s parameter Θ .
until converge
return model $\mathcal{M}$ and prediction results $\{\hat{y}_i\}$.

to further encode the behavior patterns that denoted by $\mathbf{o}_u \in \mathbb{R}^2$ and the prediction results $\hat{y}_u$ for node u could be obtained by

$$\hat{y}_u = \text{softmax}(\mathbf{o}_u). \quad (10)$$

Finally, the model could be optimized with binary cross entropy as the objective

$$\mathcal{L} = \frac{1}{N} \sum_{u \in \mathcal{V}} y_u \log(\hat{y}_u) + (1 - y_u) \log(1 - \hat{y}_u). \quad (11)$$

E. Complexity Analysis

We give computational analysis in this section. The computational complexity of pattern learning mainly comes from three aspects: the short-term pattern learning, the long-term pattern learning, and the final classification module. The short-term learning includes multiple attention module whose complexity is $\mathcal{O}(2|V|^2D + \mathcal{E}_{\mathcal{T}}D'^2)$. The long-term pattern learning module mainly contains a GRU and neighborhood aggregation operation whose complexity is $\mathcal{O}(3D'^3 + |\mathcal{V}D'|)$. We use two-layer graph convolution in the final classification module, and its complexity is $\mathcal{O}(6|\mathcal{E}|^2D'F)$ where $\mathcal{E}$ denotes the number of edges in $\mathcal{G}$ without taking time into consideration and F is the hidden dimension of the GCN module.

IV. EXPERIMENT

In this section, we compare the proposed method with six state-of-the-art baselines on two datasets extracted from real scenarios and one synthetic dataset.

TABLE I
BASIC STATISTICS OF THE TWO DATASETS

Dataset	$ \mathcal{V} $	$ \mathcal{E}_{\mathcal{T}} $	No. Fraudsters	Anomaly Ratio
Telecom	11 427	20 581	116	1.02%
Ether	26 370	50 998	684	2.59%
Syn-Ether	26 370	51 898	200	0.76%

A. Dataset

Though fraud detection is of practical significance, real-world labeled data are hard to obtain. Previous research either manually inject anomalies into networks or regard the nodes belonging to the minority class as anomalies. Such anomaly definition might differ from the problems in practice. In this article, we evaluate the proposed method on the following two real-world datasets and one synthetic dataset, and their statistics are summarized in Table I.

- 1) *Telecom*: Telecom is a dynamic network that consists of 20 581 telecom records among 11 427 phone numbers. The telecom records range from 25 August 2022 to 30 September 2022 and 116 phone numbers are labeled as fraudsters.
- 2) *Ether* [11]: Ether is one of the largest cryptocurrency in the world second to Bitcoin. The network consists of 26 370 accounts and involves 50 998 ether transactions between 1 January 2018 and 30 June 2018. In the network, 684 accounts are labeled as phishers which are regarded as anomalous nodes.
- 3) *Syn-Ether*: We manually inject anomalous structures into the Ether transaction network following. Specifically, following [56], we randomly select m accounts in the transaction network and then connect each pair of accounts with virtual transactions if they have no history of transaction. It makes the accounts form a fully connected anomalous structure, and the m accounts are regarded as fraudulent accounts. We repeat the injection process k times and finally obtain mk virtual fraudulent accounts. In this article, we set $m = 10$ and $k = 20$.

In contrastive pretraining, every node within the input graph is involved in the pretraining process, eliminating the need to separate nodes into training and test sets. However, for supervised fine-tuning, we categorize the nodes into training and test sets in a 4:1 ratio.

B. Baseline

We compare the proposed MPIN with five graph representation learning approaches which include both static and dynamic network embedding methods.

- 1) *Node2vec* [29] is a random walk-based node embedding approach for static networks. It performs a biased random walk to combine depth-first and breadth-first searches on networks to capture network structure.
- 2) *GCN* [30] is regarded as a low-pass filter that passes the features along with edges and aggregates neighbor node features of target nodes.

TABLE II
PERFORMANCE OF DIFFERENT METHODS WITH RESPECT TO AUC, PRECISION, RECALL, AND F1 SCORE

Dataset	Metric	Node2vec	GCN	CTDNE	EvolveGCN	DGI	CoLA	MPIN
Telcom	AUC	75.69 ± 2.79	99.26 ± 0.24	82.37 ± 2.13	99.13 ± 0.43	81.70 ± 2.41	85.85 ± 1.84	99.67 ± 0.20
	Precision	28.77 ± 2.54	51.70 ± 5.44	42.98 ± 1.89	54.38 ± 2.51	66.33 ± 4.79	71.82 ± 2.15	90.78 ± 3.96
	Recall	35.29 ± 3.19	75.53 ± 1.97	51.09 ± 2.37	78.29 ± 1.22	60.88 ± 4.51	69.73 ± 2.47	75.33 ± 3.00
	F1 Score	32.71 ± 3.28	61.30 ± 2.93	48.69 ± 1.94	65.73 ± 1.47	63.42 ± 3.25	71.33 ± 2.24	88.32 ± 2.52
Ether	AUC	92.83 ± 2.50	96.89 ± 0.82	96.29 ± 0.44	98.31 ± 0.53	97.70 ± 1.17	95.39 ± 2.19	99.12 ± 0.28
	Precision	38.28 ± 5.58	66.39 ± 0.99	62.28 ± 3.75	69.23 ± 1.67	69.19 ± 4.14	68.73 ± 1.71	70.23 ± 1.12
	Recall	54.28 ± 4.57	69.38 ± 3.07	70.88 ± 3.86	72.47 ± 1.15	84.78 ± 2.42	85.29 ± 1.91	89.49 ± 1.17
	F1 Score	44.83 ± 3.87	67.83 ± 1.37	66.28 ± 2.65	71.37 ± 1.34	77.85 ± 2.09	77.35 ± 1.77	78.69 ± 2.33
Syn-Ether	AUC	76.07 ± 4.05	94.18 ± 1.01	74.39 ± 5.45	98.28 ± 1.59	99.95 ± 0.57	99.95 ± 0.87	99.97 ± 0.08
	Precision	21.33 ± 3.67	55.84 ± 2.98	30.55 ± 2.28	75.34 ± 1.87	96.08 ± 1.67	96.58 ± 1.48	95.91 ± 1.53
	Recall	22.81 ± 2.31	62.35 ± 2.65	41.01 ± 2.56	69.57 ± 1.34	92.19 ± 1.82	92.57 ± 1.29	93.50 ± 1.50
	F1 Score	21.92 ± 1.77	58.88 ± 2.51	34.97 ± 1.78	73.84 ± 1.61	92.64 ± 1.61	94.31 ± 1.43	94.68 ± 1.52

Note: The values in bold font denote the best results.

- 3) *CTDNE* [37] focuses on continuous-time dynamic networks and extends random walk with temporal bias to obtain node representations.
- 4) *EvolveGCN* [40] applies RNNs to learn the temporal correlations of GCNs' weights to map nodes in dynamic networks to lower dimensional spaces.
- 5) *DGI* [46] maximizes the mutual information between patch representations and high-level graph summaries to learn graph embeddings.
- 6) *CoLA* [50] applies contrastive learning on node-subgraph instances and provides anomaly score for nodes in attributed networks.

When implementing MPIN, we set the hidden dimension of both the attention module and the GRU to 128. We employ two fully connected layers with 256 and two hidden units to make the final prediction. In the contrastive pretraining phase, we set $\lambda = 0.1$, $\beta = 0.4$, and the behavior pattern learning module at the learning rate of $1e-3$. The learning rate for subsequent supervised learning is also set to $1e-3$. All the experiments were conducted on a computer with Ubuntu 20.04 OS and one NVIDIA GeForce RTX 3090Ti (24 GB memory) GPU.

C. Performance Comparison

To evaluate the performance of MPIN, we compare it with six graph embedding algorithms. Table II shows the average performance of MPIN as well as the baselines in terms of AUC, precision, recall, and F1 score. The best ones are marked in black font. As is shown, MPIN has the best performance, including high detection performance and low deviation, among all approaches with regard to all four metrics. The results prove the effectiveness of the characterization of three interaction patterns and contrastive pretraining. Taking all the results on the three datasets into consideration, MPIN outperforms node2vec with a notable statistical significance, as indicated by a p-value of less than 0.05. For detailed insights and supporting data, please refer to the Appendix (see the supplementary material).

No surprisingly, node2vec which is designed for static network embedding performs the worst in most cases. It is because the anomalies are rare in networks which could fail the structure characterization achieved by random walk-based

TABLE III
PERFORMANCE OF MPIN WITHOUT $\mathbf{h}^{\text{in}}$, $\mathbf{h}^{\text{out}}$, OR $\mathbf{h}^m$

		w/o $\mathbf{h}^{\text{in}}$	w/o $\mathbf{h}^{\text{out}}$	w/o $\mathbf{h}^m$
Telecom	AUC	97.58	96.32	94.89
	Precision	88.23	86.91	81.09
	Recall	72.65	73.04	70.31
	F1 Score	79.83	79.37	75.97
Ether	AUC	97.31	97.05	95.43
	Precision	68.29	67.50	63.37
	Recall	86.35	85.92	81.03
	F1 Score	76.53	75.60	71.65
Syn-Ether	AUC	98.95	98.42	96.36
	Precision	93.45	92.73	87.26
	Recall	92.17	91.63	89.85
	F1 Score	92.81	91.97	88.54

methods. The two-stage detection scheme further lowers the detection performance. Nevertheless, CTDNE, another random walk-based method, has better detection performance compared with node2vec, which is due to the temporal bias adopted in its walking strategy. The usefulness of temporal information is also shown by comparing the performance of GCN and EvolveGCN. As is described in Section III, MPIN adopts contrastive pretraining before anomaly identification. Comparing the performance of MPIN with DGI and CoLA, we could draw the conclusion that MPIN has a better ability to characterize both structural and temporal features.

Apart from Telecom and Ether, which are extracted from real-world scenarios where frauds happen, we also manually inject structural anomalies into Ether to compare the performance of different methods against such anomalies. As the results suggest, MPIN is more capable of detecting the manually injected anomalies than real-world anomalies, and so do the baseline methods such as GCN and DGI. The two random walk-based methods, node2vec and CTDNE, could not seem to capture the anomalous structure feature, resulting the poor detection performance. The results on real-world datasets and the synthetic dataset demonstrate the significance of real-world datasets in the research of graph anomaly detection since the methods evaluated on synthetic datasets may not suit real-world scenarios.

TABLE IV
PERFORMANCE OF MPIN WITH AND
WITHOUT PRETRAINING

		Telecom	Ether	Syn-Ether
AUC	MPIN-p	99.32	99.24	99.51
	MPIN	99.67	99.12	99.97
	GAIN	0.35%	-0.12%	0.46%
Precision	MPIN-p	83.79	64.17	88.72
	MPIN	90.78	70.23	95.91
	GAIN	8.34%	9.44%	8.10%
Recall	MPIN-p	68.15	88.39	89.03
	MPIN	75.33	89.49	93.50
	GAIN	10.54%	1.24%	5.02%
F1 Score	MPIN-p	74.17	76.36	87.93
	MPIN	88.32	93.34	94.68
	GAIN	19.08%	22.24%	7.68%

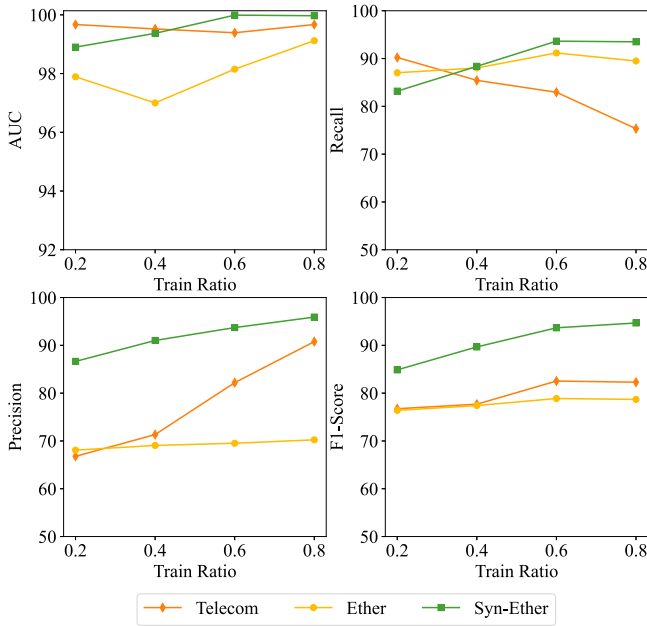


Fig. 4. Performance of MPIN with the increment of training data.

D. Ablation Study

1) *The Influence of Different Patterns*: As described above, MPIN takes three types of interactions, i.e., inflows, outflows, and their mutual influence, into consideration. The notable performance shown in Table II has proved the effectiveness of MPIN. A subsequent question arises: which type of interaction information is most important in fraudster identification? To investigate into question, we drop the inflow pattern, the outflow pattern, and their mutual influence in MPIN, respectively. As presented in Table III, the model without h^m behaves the worst in anomaly detection over the three datasets, which indicates that the mutual influence between inflows and outflows has the most impact on model inference. It is might because that h^m contains part of the information of h^{in} and h^{out} , making it the most informative one.

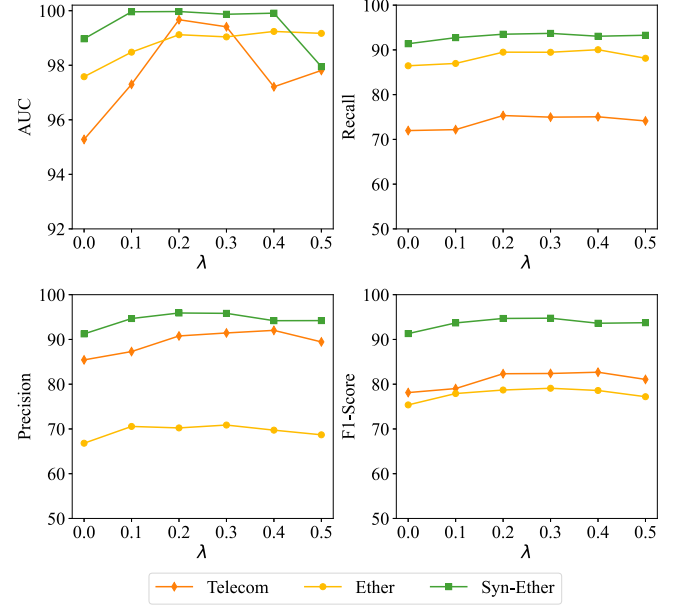


Fig. 5. Performance of MPIN as λ varies.

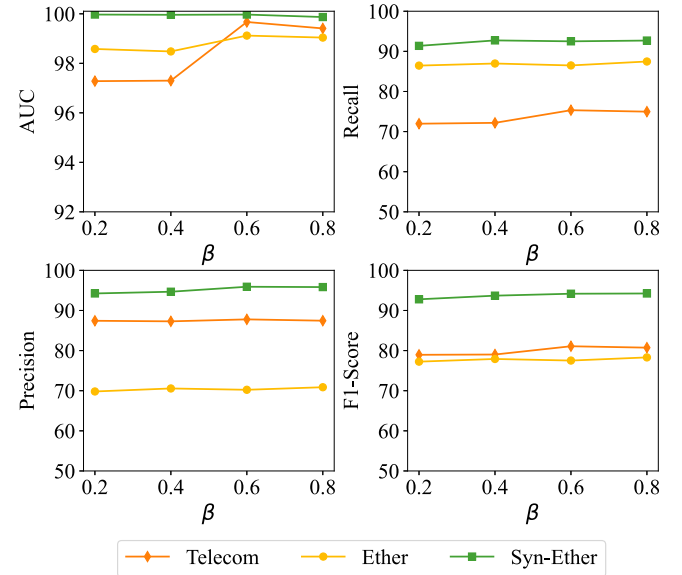


Fig. 6. Performance of MPIN as β varies.

2) *The Effectiveness of Pretraining*: Since the scheme of MPIN contains contrastive pretraining, we would like to investigate how much the pretraining contributes to the anomaly detection. Table IV presents the performance of MPIN with and without contrastive pretraining (denoted by MPIN-p). The results show that the pretraining could indeed improve the detection performance in most cases. The AUC value of MPIN always stays at a high level. The unnoticeable performance decrease of MPIN with pretraining on Ether might be due to the noise in model training.

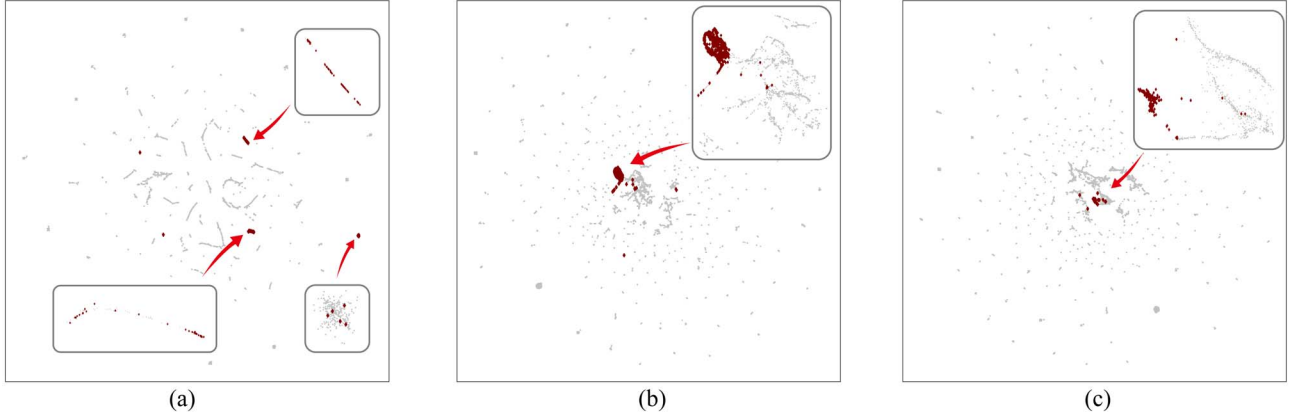


Fig. 7. Behavior pattern visualization of Comm, Ether, and Syn-Ether. The red $\blacklozenge$ represents fraudsters, and the dots in light grey are normal nodes. The images in rounded rectangles are enlarged versions of fraudulent clusters.

E. Parameter Analysis

1) *The Influence of the Amount of Training Data:* In this article, fraud detection is considered as a classification problem with extreme class imbalance. Thus, we would like to further investigate into the performance of MPIN with labeled data reduction. With curiosity, We vary the percentage of training data from 20% to 80% and reevaluate MPIN on the three datasets. Fig. 4 shows the performance of MPIN with the increment of training data. The results on the three datasets represent three different situations. On Ether, the performance of MPIN does not significantly improve when training data are added, which shows that MPIN indeed learns the patterns of labeled fraudsters and again proves the effectiveness of the proposed model. Compared with the performance on Ether, MPIN gradually performs better with the increment of training data, which accords with our intuition. In this situation, MPIN needs enough labeled data to perfect its detection ability. As for Telecom, we find that the tendency of precision is as opposed to that of recall, while AUC and F1 score do not change significantly. With such a small amount of true labeled fraudsters, MPIN is easy to find false-positive sample, resulting in the high recall and low precision at the beginning. More labeled data could rectify its judgment.

2) *The Influence of λ :* In the contrastive pretraining, we adopt the contrastive loss $\mathcal{L}_c$ to learn distinct transaction patterns and the local tightness loss $\mathcal{L}_t$ to gather the accounts with similar transaction patterns. We would like to explore the influences of the two objectives on the final detection performance. As we can observe from Fig. 5, the addition of $\mathcal{L}_t$ indeed improves the performance of MPIN. As λ increases, the average performance gets better and the deviation gets smaller. When $\lambda > 0.3$, both precision and recall decline. We argue that it is because the model focuses more on gathering accounts into local clustering rather than learning distinct transaction patterns when λ gets too large. Based on the experimental results, we suggest that $[0.1, 0.3]$ would be a proper range for λ .

3) *The Influence of β :* β controls how many edges will be removed to generate positive samples. To quantify the influence that β has on MPIN's performance, we vary it from 0.2 to 0.8

and observe the performance change. The results are presented in Fig. 6. We found that the performance of MPIN does not significantly vary when β changes. It might be because there are multiple parallel temporal edges and only removing part of them still could generate effective positive samples. With the help of soft-positive samples, the performance change could be even smaller. Empirically, $[0.2, 0.8]$ would be an appropriate range for λ , especially for the three datasets.

F. Behavior Pattern Visualization

Besides pursuing the performance of graph anomaly detection, we also visualize¹ the behavior patterns of individuals in different networks to investigate which kind of patterns the model learns when performing anomaly detection. As presented in Fig. 7, the nodes in all three datasets share similar interaction patterns cluster together. Specifically, the fraudsters in Telecom are distributed in three clusters while normal phone numbers possess similar communication patterns from many other clusters. As for Ether and Syn-Ether, there is only one main cluster formed by fraudsters in the networks. It indicates that MPIN indeed learns exact interaction patterns. Meanwhile, we should notice that some anomalous nodes and normal nodes are mixed in one cluster, which may lead to the misjudgment of fraudsters.

V. CONCLUSION

In this article, we proposed an MPIN to address the problem of fraudulent account identification in transaction networks. Considering inflows, outflows, and the mutual influence of the two kinds of interaction from both short-term and long-term perspectives, MPIN could well learn the behavior patterns of each account and then distinguish the fraudulent accounts from normal ones. The contrastive learning process indeed stabilized the learning process and led to a low-variance performance. The experimental results on two real-world transaction networks proved the superior performance of MPIN compared with several baselines. By now, the fraudulent account identification

¹The visualization is implemented by UMAP: <https://github.com/lmcinnes/umap>.

has been achieved through supervised training. Meanwhile, we performed fraud detection on the whole graph which may limit the scalability of large-scale networks with millions of nodes. The contrastive-pretraining was actually time-costly for the whole graph inference. In future works, we would like to explore the ways, e.g., inference based on subgraphs, to reduce the computation cost while maintaining the detection accuracy.

REFERENCES

- [1] Q. Tao, G. W. Wu, et al., "Posterior probability support vector machines for unbalanced data," in *IEEE Transactions on Neural Networks*, vol. 16, no. 6, Nov. 2005, pp. 1561–1573.
- [2] F. Y. Wang, D. Zeng, et al., "A study of the human flesh search engine: Crowd-powered expansion of online knowledge," *Computer*, vol. 43, no. 8, pp. 45–53, Aug. 2010.
- [3] Y. Chen, Y. Lv, et al., "Traffic flow imputation using parallel data and generative adversarial networks," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 4, Apr. 2020, pp. 1624–1630.
- [4] W. Wang, X. Na, et al., "Decision-making in driver-automation shared control: A review and perspectives," in *IEEE/CAA Journal of Automatica Sinica*, vol. 7, no. 5, Sep. 2020, pp. 1289–1307.
- [5] J. Cao, K. Zeng, et al., "Web-based traffic sentiment analysis: Methods and applications," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 15, no. 2, Apr. 2014, pp. 844–853.
- [6] Y. Xing, C. Lv, et al., "Identification and analysis of driver postures for in-vehicle driving activities and secondary tasks recognition," in *IEEE Transactions on Computational Social Systems*, vol. 5, no. 1, Mar. 2018, pp. 95–108.
- [7] J. Han et al., "ParaDefender: A scenario-driven parallel system for defending metaverses," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 53, no. 4, pp. 2118–2127, Apr. 2023.
- [8] M. del Mar Roldán-García, J. García-Nieto, and J. F. Aldana-Montes, "Enhancing semantic consistency in anti-fraud rule-based expert systems," *Expert Syst. Appl.*, vol. 90, pp. 332–343, Dec. 2017.
- [9] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [10] D. Wang et al., "A semi-supervised graph attentive network for financial fraud detection," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 598–607.
- [11] J. Wu et al., "Who are the phishers? Phishing scam detection on Ethereum via network embedding," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 52, no. 2, pp. 1156–1166, Feb. 2022.
- [12] Y. Xie, G. Liu, C. Yan, C. Jiang, and M. Zhou, "Time-aware attention-based gated network for credit card fraud detection by extracting transactional behaviors," *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 3, pp. 1004–1016, Jun. 2023.
- [13] P. K. Verma, P. Agrawal, I. Amorim, and R. Prodan, "WELFake: Word embedding over linguistic features for fake news detection," *IEEE Trans. Comput. Social Syst.*, vol. 8, no. 4, pp. 881–893, Aug. 2021.
- [14] A. C. Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten, "Feature engineering strategies for credit card fraud detection," *Expert Syst. Appl.*, vol. 51, pp. 134–142, Jun. 2016.
- [15] Y. Xie, G. Liu, C. Yan, C. Jiang, M. Zhou, and M. Li, "Learning transactional behavioral representations for credit card fraud detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5735–5748, Apr. 2024.
- [16] R. Li, Z. Liu, Y. Ma, D. Yang, and S. Sun, "Internet financial fraud detection based on graph learning," *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 3, pp. 1394–1401, Jun. 2023.
- [17] X. Zhang, Y. Han, W. Xu, and Q. Wang, "HOBAs: A novel feature engineering methodology for credit card fraud detection with a deep learning architecture," *Inf. Sci.*, vol. 557, pp. 302–316, May 2021.
- [18] S. Han, K. Zhu, M. Zhou, and X. Cai, "Information-utilization-method-assisted multimodal multiobjective optimization and application to credit card fraud detection," *IEEE Trans. Comput. Social Syst.*, vol. 8, no. 4, pp. 856–869, Aug. 2021.
- [19] S. Han, K. Zhu, M. Zhou, and X. Cai, "Competition-driven multimodal multiobjective optimization and its application to feature selection for credit card fraud detection," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 52, no. 12, pp. 7845–7857, Dec. 2022.
- [20] H. Zhu, M. Zhou, G. Liu, Y. Xie, S. Liu, and C. Guo, "NUS: Noisy-sample-removed undersampling scheme for imbalanced classification and application to credit card fraud detection," *IEEE Trans. Comput. Social Syst.*, vol. 11, no. 2, pp. 1793–1804, Apr. 2024.
- [21] H. Zhu, G. Liu, M. Zhou, Y. Xie, A. Abusorrah, and Q. Kang, "Optimizing weighted extreme learning machines for imbalanced classification and application to credit card fraud detection," *Neurocomputing*, vol. 407, pp. 50–62, Sep. 2020.
- [22] D. Xi et al., "Modeling the field value variations and field interactions simultaneously for fraud detection," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 17, 2021, pp. 14957–14965.
- [23] Z. Xiao and J. Jiao, "Explainable fraud detection for few labeled time series data," *Secur. Commun. Netw.*, vol. 2021, Jun. 2021, Art. no. 9942464.
- [24] N. Kant and M. Mahajan, "Time-series outlier detection using enhanced k-means in combination with PSO algorithm," in *Proc. Eng. Vib. Commun. Inf. Process*, Singapore: Springer, 2019, pp. 363–373.
- [25] Z. Liu, C. Chen, X. Yang, J. Zhou, X. Li, and L. Song, "Heterogeneous graph neural networks for malicious account detection," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage.*, 2018, pp. 2077–2085.
- [26] D. Cheng, Y. Tu, Z.-W. Ma, Z. Niu, and L. Zhang, "Risk assessment for networked-guarantee loans using high-order graph attention representation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 5822–5828.
- [27] B. Xu, H. Shen, B. Sun, R. An, Q. Cao, and X. Cheng, "Towards consumer loan fraud detection: Graph neural networks with role-constrained conditional random field," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 5, 2021, pp. 4537–4545.
- [28] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery & Data Mining*, 2014, pp. 701–710.
- [29] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery & Data Mining*, 2016, pp. 855–864.
- [30] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017.
- [31] W. Lu, N. Jiang, D. Jin, H. Chen, and X. Liu, "Learning distinct relationship in package recommendation with graph attention networks," *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 6, pp. 3308–3320, Dec. 2023.
- [32] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [33] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery & Data Mining*, 2019, pp. 257–266.
- [34] G. Li, M. Muller, A. Thabet, and B. Ghanem, "DeepGCNs: Can GCNs go as deep as CNNs?" in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9267–9276.
- [35] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7436–7456, Nov. 2022.
- [36] C. He, X. Fei, Q. Cheng, H. Li, Z. Hu, and Y. Tang, "A survey of community detection in complex networks using nonnegative matrix factorization," *IEEE Trans. Comput. Social Syst.*, vol. 9, no. 2, pp. 440–457, Apr. 2022.
- [37] G. H. Nguyen, J. B. Lee, R. A. Rossi, N. K. Ahmed, E. Koh, and S. Kim, "Continuous-time dynamic network embeddings," in *Proc. Companion Proc. Web Conf.*, 2018, pp. 969–976.
- [38] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery & Data Mining*, 2018, pp. 2672–2681.
- [39] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, "DyRep: Learning representations over dynamic graphs," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [40] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, 2020, pp. 5363–5370.
- [41] J. Chen et al., "E-LSTM-D: A deep learning framework for dynamic network link prediction," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 51, no. 6, pp. 3699–3712, Jun. 2021.

- [42] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.
- [43] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 1597–1607.
- [44] Y. Jian, C. Gao, and S. Vosoughi, "Non-linguistic supervision for contrastive learning of sentence embeddings," vol. 35, 2022, pp. 3533–35548.
- [45] R. D. Hjelm et al., "Learning deep representations by mutual information estimation and maximization," in *7th Int. Conf. Learn. Representations, ICLR*, New Orleans, LA, USA, May 2019. OpenReview.net, 2019. [Online]. Available: <https://openreview.net/forum?id=Bklr3j0cKX>
- [46] P. Veličković, W. Fedus, W. L. Hamilton, P. Lio, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *7th Int. Conf. Learn. Representations, ICLR*, New Orleans, LA, USA, May 2019. OpenReview.net, 2019. [Online]. Available: <https://openreview.net/forum?id=rklz9iAcKQ>
- [47] Y. Ren and B. Liu, "Heterogeneous deep graph infomax," 2019, *arXiv:1911.08538*.
- [48] F. L. Opolka, A. Solomon, C. Cangea, P. Veličković, P. Lio, and R. D. Hjelm, "Spatio-temporal deep graph infomax," 2019, *arXiv:1904.06316*.
- [49] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 5812–5823.
- [50] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis, "Anomaly detection on attributed networks via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2378–2392, Jun. 2022.
- [51] H. Yang, H. Chen, S. Pan, L. Li, P. S. Yu, and G. Xu, "Dual space graph contrastive learning," in *Proc. ACM Web Conf.*, 2022, pp. 1238–1247.
- [52] S. Wan, S. Pan, J. Yang, and C. Gong, "Contrastive and generative graph convolutional networks for graph-based semi-supervised learning," in *Proc. 35th AAAI Conf. Artif. Intell.*, Palo Alto, CA, USA: AAAI Press, 2021, pp. 10 049–10 057.
- [53] J. Zeng and P. Xie, "Contrastive self-supervised learning for graph classification," in *Proc. 35th AAAI Conf. Artif. Intell.*, AAAI Press, 2021, pp. 10824–10832.
- [54] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, *arXiv:1807.03748*.
- [55] S. Li, W. Li, A. M. Luvembe, and W. Tong, "Graph contrastive learning with feature augmentation for rumor detection," *IEEE Trans. Comput. Social Syst.*, early access, doi: 10.1109/TCSS.2023.3269303.
- [56] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. SIAM Int. Conf. Data Mining.*, Calgary, Alberta, Canada: SIAM, 2019, pp. 594–602.



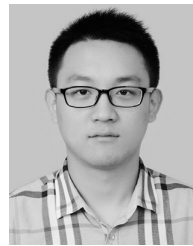
Manzhi Yang received the master's degree in cryptography from Beijing University of Posts and Telecommunications, Beijing, China, in 2005. He is currently working toward the Ph.D. degree in intelligent science and systems engineering with Macao University of Science and Technology, Macao, China.

He is a Senior Expert in network security systems with Eversec Technology Company Ltd., Beijing, China. His research interests include parallel intelligence, parallel security, and cybersecurity.



Jian Zhang received the B.S. degree in automation and the Ph.D. degree in control theory and engineering from Zhejiang University of Technology, Hangzhou, China, in 2017 and 2022, respectively.

He is currently a Lecturer with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His research interests include graph neural networks, machine learning security, and anomaly detection.



Liyuan Lin received the B.S. degree in computer science from China Jiliang University, Hangzhou, China, in 2021. He is currently working toward the M.S. degree in cyberspace security from Hangzhou Dianzi University, Hangzhou, China.

His research interests include graph representation learning and anomaly detection.



Jinpeng Han (Student Member, IEEE) received the B.E. degree in automation from the School of Automation, Wuhan University of Technology, Wuhan, China in 2016. He is currently working toward the Ph.D. degree in software engineering with the School of Software Engineering, Xi'an Jiaotong University, Xi'an, China, as well as the State Key Laboratory for Management and Control of Complex Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China.

His research interests include parallel intelligence, parallel security, and security games.



Xiaoguang Chen received the M.E. degree in information security from Beijing University of Posts and Telecommunications, Beijing, China, in 2008. He is currently working toward the Ph.D. degree in intelligent science and systems engineering with Macao University of Science and Technology, Macao, China.

He is specialized in industrial IOT security systems with Eversec Technology Company Ltd., Beijing, China. His research interests include parallel intelligence, parallel security, and cybersecurity.



Zhen Wang received the B.Sc., M.Eng., and Ph.D. degrees in software engineering from Dalian University of Technology, Dalian, China, in 2007, 2009, and 2016, respectively.

He is a Professor with the School of Cyberspace and the Experimental Center of Data Science and Intelligent Decision-Making, Hangzhou Dianzi University, Hangzhou, China. From 2014 to 2016, he was a Research Fellow with Nanyang Technological University, Singapore. His research interests include network security, artificial intelligence security, complex networks, reinforcement learning, and algorithmic game theory.



Fei-Yue Wang (Fellow, IEEE) received the Ph.D. degree in computer and systems engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 1990.

He joined The University of Arizona, Tucson, USA, in 1990, and became a Professor and the Director of the Robotics and Automation Laboratory and the Program in Advanced Research for Complex Systems. In 1999, he founded the Intelligent Control and Systems Engineering Center, Institute of Automation, Chinese Academy of Sciences

(CAS), Beijing, China, under the support of the Outstanding Chinese Talents Program from the State Planning Council, and in 2002, was appointed as the Director of the Key Laboratory of Complex Systems and Intelligence Science, CAS, and the Vice President of Institute of Automation, CAS, in 2006.

He founded CAS Center for Social Computing and Parallel Management, in 2008, and became the State Specially Appointed Expert and the Founding Director of the State Key Laboratory for Management and Control of Complex Systems, in 2011. His research interests include methods and applications for parallel intelligence, social computing, and knowledge automation.

He is a fellow of INCOSE, IFAC, ASME, and AAAS. In 2007, he received the National Prize in Natural Sciences of China, numerous best papers awards from IEEE Transactions, and became an Outstanding Scientist of ACM for

his work in intelligent control and social computing. He received the IEEE ITS Outstanding Application and Research Awards in 2009, 2011, and 2015, respectively, the IEEE SMC Norbert Wiener Award in 2014, and became the IFAC Pavel J. Nowacki Distinguished Lecturer in 2021. Since 1997, he has been serving as the General or Program Chair of over 30 IEEE, INFORMS, IFAC, ACM, and ASME conferences. He was the President of the IEEE ITS Society from 2005 to 2007, the IEEE Council of RFID from 2019 to 2021, the Chinese Association for Science and Technology, USA, in 2005, the American Zhu Kezhen Education Foundation from 2007 to 2008, the Vice President of the ACM China Council from 2010 to 2011, the Vice President and the Secretary General of the Chinese Association of Automation from 2008 to 2018, the Vice President of IEEE Systems, Man, and Cybernetics Society from 2019 to 2021. He was the Founding Editor-in-Chief (EiC) of *International Journal of Intelligent Control and Systems* from 1995 to 2000, *IEEE Intelligent Transportation Systems Magazine* from 2006 to 2007, *IEEE/CAA JOURNAL OF AUTOMATICA SINICA* from 2014 to 2017, China's *Journal of Command and Control* from 2015 to 2021, and *Chinese Journal of Intelligent Science and Technology* from 2019 to 2021. He was the EiC of IEEE INTELLIGENT SYSTEMS from 2009 to 2012, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS from 2009 to 2016, and IEEE TRANSACTIONS ON COMPUTATIONAL SOCIAL SYSTEMS from 2017 to 2020. Currently, he is the President of CAA's Supervision Council and the EiC of IEEE TRANSACTIONS ON INTELLIGENT VEHICLES.